Replanning on News

Public Announcements during the Execution of Epistemic Policies, with One, Two and Three Robots, on ePlanSys

Haniel Ulises

Epistemic Robotics

6 November 2026

A courier robot must carry a resident's medicine across a residence at night, through one of three fire doors, any of which may be shut. It may cross only a door it knows to be open. Its policy, computed by Aletheia over an EPDDL task and executed by ePlanSys in Gazebo, branches on what the robots see and say. While it runs, information arrives that no policy accounted for: a door panel's report, a maintenance note relating two doors, a message from a cleaning robot cut off before its answer. The news enters the executor as a public announcement into its epistemic state; the running action checks the belief on every tick, fails, and the mission plans again from the model the announcement leaves. The team grows across three problems, from the courier alone to three robots, and so does the kind of news: a literal, a relation between doors, and a statement about what one robot knows that changes no fact of any door. The formal analysis proves that a policy over positive preconditions remains a solution after any truthful announcement, under a condition on the messages it sends, and gives a four-world counterexample in which news breaks a policy by neutralising what the occurrence of a message would have revealed. It follows that planning again never costs worst-case depth, and that news which leaves the set of valuations unchanged is invisible to every replanner over facts. Depth is not time: a first batch showed the policy of least depth after the cleaner's message starting a drive before the answer that decides it, and losing time to the policy kept; committing drives late, a transformation proved to preserve solutions and depth, removed the loss. In 128 Gazebo runs over 32 seeds, each under four formulations, every epistemic run delivered; planning again on the news saved 27.4 and 15.6 s on average against the policy kept in the first two acts, in every pair, and 2.2 s in the third, not distinguishable from nothing and concentrated in the one world where the cleaner's answer spares a drive. Planning again on a tautology announced at the same instant gained nothing.

Contents

1 Scope **2**

2 The residence **2**

3 The planning task **3**
3.1 Models, events and announcements . . . 3
3.2 The domain . . . 4
3.3 Three problems, three kinds of news . . . 4
3.4 The policies . . . 4

4 How news enters an executing policy **5**

5 Analysis **5**
5.1 The positive fragment . . . 5
5.2 Persistence, and when it fails . . . 6
5.3 Planning again never costs depth . . . 7
5.4 Depth is not time: committing late . . . 7
5.5 News invisible to replanning over facts . . . 8

6 The batch **8**

7 Implementation findings **10**

8 Limitations **11**

1 Scope

A policy for a partially observable domain is planned against a model of what the agents know. Execution can change that model in two ways. The first is planned for: a sensing action selects one of the continuations the policy holds for its outcomes. The second is not: information reaches the agents from outside the plan, from an operator, a building's door panel or another team, at a moment the plan did not choose. This report concerns the second.

The question is what such information is worth to a team that plans over knowledge, and when it can be used at all. Three questions are separated. Whether a policy remains correct after the news (Section 5.2): if it does, a mission may ignore the news, and replanning buys only cost. Whether planning again pays, and what part of the gain is the news and what part is merely planning again (Sections 5.3 and 6). And which news a replanner over facts, the classical alternative, can use (Section 5.5).

The setting is a residence at night, built for this purpose in Gazebo with the AWS RoboMaker residential models [1], and three planning problems of growing team size. The mechanism exercised is ePlanSys's: the epistemic state node's `announce` service, the `CheckBeliefUnchanged` condition the executor's behaviour tree carries inside every running action, and the in-process epistemic solver, which plans from the model the state publishes.

2 The residence

The residence is a single storey of 24 by 16 m (Figure 1). The south half holds a lounge, a dining room and the nurses' station, all opening onto a hall that runs the width of the building. North of the hall a band of linen rooms and a kitchenette, 2.6 m deep, carries the fire doors: three passages cross it, each closed by a door near its north end. The west and east passages are straight, so a robot at a passage's mouth sees the door down its length. The middle passage turns twice, so its door cannot be seen from its mouth; the kitchenette beside it has a hatch onto the passage's last leg, and from the hatch the door can be seen. North of the band a corridor runs the width of the building and four residents' rooms open off it; the resident is in the north-west room.

A door is a leaf of AWS's door model, cut down to the walls' height, standing across its passage when shut and swung against the corridor's wall when open. The furniture is AWS's: beds, wardrobes, sofas, a dining table and chairs, a cooking bench, refrigerators. What a robot can hit and what its laser sees is the collision mesh, and the floor plan the tools reason over is taken from those meshes, cut at the heights of a TurtleBot3's body and laser (`tools/furniture.py`).

Robots. Three TurtleBot3 Burgers, each with its laser's range set to 8 m: the courier c, which carries the medicine, the scout s, which waits at the hatch, and the cleaning robot k, parked in the corridor after its round, which has a radio and does not drive. The problems use the first one, two and three of them.

Seeing a door. A robot decides a door from its own laser while it stands within 0.6 m of a place the door is seen from. The returns are put in the world frame by the robot's odometry; returns on the shut leaf's rectangle, at least five, say shut, and returns beyond the door line with none on the leaf say open. Rays cast over the grid in all eight states of the doors confirm what the domain assumes (`tools/check_views.py`): each straight passage's mouth decides its door, the hatch decides the middle door, and from the middle passage's mouth the scan is identical with the door open or shut.

Driving. A drive runs on a roadmap: the south half as far as each door, the corridor, and each crossing on a fixed route through its own passage. A drive may begin anywhere, since a policy abandoned on news leaves a robot part of the way along a drive; it is routed from whichever end of the roadmap's nearest edge makes the route shortest. The controller is a dynamic window over



Figure 1: The residence from above, drawn from the layout and the furniture's collision meshes, in the state where the west door is open and the others shut. Tinted: the three passages. Rings: the places a door is seen from, the mouths of the west and east passages and the kitchenette's hatch. The courier starts at the nurses' station, the scout at the hatch, the cleaner in the corridor's east end.

the robot's own laser. When the executor cancels a drive the base is stopped at once. Every route keeps 0.52 m from walls and furniture.

3 The planning task

3.1 Models, events and announcements

Definition 1 (Epistemic model and state)**.** Given agents $\mathcal{A}$ and atoms P, an *epistemic model* is $\mathcal{M} = (W, (R_i)_{i\in\mathcal{A}}, V)$ with worlds W, an equivalence relation R_i per agent and a valuation $V : W \to 2^P$. A *state* is $s = (\mathcal{M}, W_d)$ with designated worlds $\emptyset \neq W_d \subseteq W$, the worlds the planner cannot rule out. $\mathcal{M}, w \models K_i\varphi$ iff $\mathcal{M}, v \models \varphi$ for all v with wR_iv; $Kw_i\varphi$ abbreviates $K_i\varphi \vee K_i\neg\varphi$; $s \models \varphi$ iff $\mathcal{M}, w \models \varphi$ for all $w \in W_d$.

Definition 2 (Event model and product update [2, 3])**.** An event model $\mathcal{E} = (E, (Q_i)_{i\in\mathcal{A}}, pre, post)$ has events E, relations Q_i, preconditions $pre(e)$ and postconditions. The product $\mathcal{M} \otimes \mathcal{E}$ has worlds $\{(w, e) : \mathcal{M}, w \models pre(e)\}$, $(w, e)R_i'(v, f)$ iff wR_iv and eQ_if, and the valuation V changed by $post(e)$. An action is an event model with designated events E_d; it is applicable in s when every $w \in W_d$ has some $e \in E_d$ with $\mathcal{M}, w \models pre(e)$, and its outcomes in s are the designated events realised at designated worlds.

Definition 3 (Public announcement [4, 3])**.** For a formula φ, $\mathcal{M}|\varphi$ is the submodel of $\mathcal{M}$ on $[\![\varphi]\!]_\mathcal{M} = \{w : \mathcal{M}, w \models \varphi\}$, and $s|\varphi = (\mathcal{M}|\varphi, W_d \cap [\![\varphi]\!]_\mathcal{M})$, defined when this intersection is not empty.

Definition 4 (Policy and solution [5, 6])**.** A policy is a finite tree whose nodes carry actions and whose edges carry outcomes. It is a *solution* from s when its root action is applicable in s, every

outcome realised in s has a child that is a solution from the resulting state, and at a leaf the goal holds. Its height $\mathrm{h}_s(\pi)$ is the length of its longest branch realised from s, and $\mathrm{h}^*(s)$ the least height of a solution from s, the quantity Aletheia's AO* minimises.

3.2 The domain

The atoms are $open_w$, $open_m$, $open_e$, the robots' places and their roles. Five kinds of action (`epddl-workspace/replanning-news`):

goto-v$d(r)$ the courier or the scout drives to the mouth of passage d; public and ontic.

look-$d(r)$ r looks at door d from where it can be seen, the middle door only from the hatch; semi-private sensing: r observes which event occurs, the others observe that r looked, not what it saw. Events e^{open} and e^{shut} with preconditions $open_d$ and $\neg open_d$.

tell-$d(r)$ r says over the radio whether door d is open; public sensing, events with preconditions $open_d \wedge Kw_r\, open_d$ and $\neg open_d \wedge Kw_r\, open_d$. The modal part is shared by both events, the shape ePlanSys's executor requires.

cross-$d(c)$ the courier crosses door d into the corridor; public and ontic, with precondition $K_c open_d$.

deliver(c) the courier drives along the corridor to the resident.

The goal is the courier with the resident. Every precondition and the goal lie in the positive fragment $\mathcal{L}^+$ of Definition 5.

3.3 Three problems, three kinds of news

In every problem at least one door is open, and nobody knows which: seven valuations. The initial states are given explicitly, every world designated.

Act 1: the courier alone. Seven worlds, one per valuation. The news, from the door panel: *the east door is shut*, the literal $\neg open_e$. It leaves three worlds.

Act 2: the courier and the scout. Seven worlds. The news, from maintenance: *the middle and east doors hang on one circuit, both open or both shut*, the relation $open_m \leftrightarrow open_e$. It leaves three worlds; with it, a look at one of the two doors settles the other.

Act 3: and the cleaner. Fourteen worlds: each valuation twice, once where the cleaner saw the east door on its round and once where it did not. No atom records the difference. Where it saw the door, k's relation separates the worlds by $open_e$; where it did not, k considers every unseen world possible. The news is a message from k that breaks off before its answer: *I have checked the east door, it is*, that is, the announcement of $Kw_k\, open_e$. It leaves seven worlds, one per valuation, and the same seven valuations.

3.4 The policies

Aletheia's AO* returns, for each problem before the news, a policy of height 7 (Table 1). In act 1 the courier looks at the east door, then the west, and crosses the middle door, if it comes to that, by elimination: nobody can see the middle door but the scout. In acts 2 and 3 the scout looks at the middle door through the hatch while the courier drives east, and says what it saw. After the news, planned from the model the announcement leaves, the policies have height 5, 5 and 6. In act 3 the new policy asks the cleaner: **tell-e**(k).

	worlds	height	worlds after	kept, height	planned after, height
act 1	7	7	3	7	5
act 2	7	7	3	7	5
act 3	14	7	7	7	6

Table 1: The policies, checked against the domain's semantics without a planner (`tools/policy_check.py`). *Kept*: the policy planned before the news, on the model the news leaves; it remains a solution in each act, as Theorem 1 predicts. Planning from that model takes 0.10 to 0.19 s.

4 How news enters an executing policy

ePlanSys executes a policy as a behaviour tree, each action a sequence whose running part is a reactive sequence of three nodes: PlanSys2's over-all condition check, `CheckBeliefUnchanged`, and the action's execution. The epistemic state node holds the pointed model and a counter, its belief version, which it publishes with the model. An action's own epistemic update does not change the counter. The `announce` service restricts the model to the worlds where the formula holds, refuses a formula false at every designated world, and increments the counter. On the next tick, `CheckBeliefUnchanged` in the running action sees a version different from the one it started with and fails; the reactive sequence halts the action, whose performer stops the robot, and the tree fails.

Every policy, the first and every one after it, passes through one step of the mission's own before it is executed: the courier's drives are committed late (Section 5.4).

The mission then asks for a policy again. The in-process solver, `plansys2/EpistemicPlanSolver`, takes its initial state from the model the state node publishes, so the new policy starts from the mission as it got: what the robots have looked at and said, and the news. The interrupted action left no effect, so a robot interrupted mid-drive is, in the model, where it set out from; its next drive starts from where it stands.

Four formulations are compared:

replan the news is announced, and the mission plans again;

null a tautology is announced at the same instant: the policy is interrupted and planned again from a model the announcement did not change;

keep the news is heard and not announced; the policy planned at the start runs to its end;

classical POPF [9] through PlanSys2 [10], replanning over the valuations consistent with what is known, in the manner of FF-Replan [8]: the prior, the news when it states something of the doors, and every door found shut. It plans in the most convenient of them, asserting open only the door whose trip from where the courier is is shortest, and plans again when the news rules that door out or the courier's laser finds it shut on the way through. It uses literal and relational news in full, and it never stops for want of a plan: a classical model that gates each crossing on a check, the knowledge precondition written as a fact, has no plan in act 1 when only the middle door is open, since nothing the courier can do checks that door.

The news arrives 14.0 s into the mission, while the courier drives towards the east door in every epistemic formulation.

5 Analysis

5.1 The positive fragment

Definition 5 (Positive fragment)**.** $\mathcal{L}^+$ is generated by literals, conjunction, disjunction and K_i. It contains Kw_ip, and every precondition and goal of the domain.

Lemma 1 (Preservation)**.** If $\mathcal{M}'$ is a submodel of $\mathcal{M}$, w a world of $\mathcal{M}'$ and $\varphi \in \mathcal{L}^+$, then $\mathcal{M}, w \models \varphi$ implies $\mathcal{M}', w \models \varphi$.

Proof. By induction on φ. Literals depend on $V(w)$ alone, which the submodel keeps; conjunction and disjunction are immediate. For $K_i\psi$: every R'_i-successor v of w in $\mathcal{M}'$ is an R_i-successor in $\mathcal{M}$, where ψ holds, so by the hypothesis ψ holds at v in $\mathcal{M}'$. □

Announcing a true formula can only add knowledge expressed positively. It can remove the truth of a negative one, which is the content of Moore's paradox and of unsuccessful updates [7]; no precondition here is negative in that sense.

5.2 Persistence, and when it fails

A message whose precondition says that the speaker knows tells the listeners more than its answer: its occurrence rules out the worlds where the speaker could not have spoken. Persistence depends on that side channel.

Definition 6 (Transparent message)**.** An action whose designated events have preconditions $pre(e) = \chi_e \wedge \mu$, with χ_e propositional and pairwise exclusive and $\mu \in \mathcal{L}^+$ shared, is executed *transparently* at a state when μ holds at every world of the state, designated or not.

A message after a semi-private look is transparent: every agent observed the look and knows that the looker knows. So is the cleaner's message after the announcement of $Kw_k open_e$. The crossing is not: where the courier does not know the door open its precondition fails, and those worlds are removed.

Theorem 1 (Persistence)**.** Let π be a solution from s whose preconditions and goal lie in $\mathcal{L}^+$, whose actions all have the shape of the definition above, and each of whose actions is executed transparently or followed, on its branch, only by propositional preconditions and goal. Then for every φ with $W_d \cap [\![\varphi]\!] \neq \emptyset$, π is a solution from $s|\varphi$.

Proof. Write t_β for the state π reaches from s along a branch β, and t'_β for the one it reaches from $s|\varphi$. We show by induction along β that, as long as every action so far was executed transparently, t'_β is the submodel of t_β on the descendants of $[\![\varphi]\!]$, with the designated worlds of t_β among them. At the root this is the definition of $s|\varphi$. Let the next action have events $pre(e) = \chi_e \wedge \mu$, executed transparently at t_β. By Lemma 1, μ holds at every world of t'_β too, so in both products a pair (w, e) exists iff χ_e holds at w, which the submodel does not change. Hence $t'_\beta \otimes \mathcal{E}$ is the submodel of $t_\beta \otimes \mathcal{E}$ on the pairs whose first component descends from $[\![\varphi]\!]$, and the designated pairs correspond. The action is applicable at t'_β, since its precondition holds at the designated worlds of t_β and so, by Lemma 1, at those of t'_β. Its outcomes at t'_β are among those at t_β: an outcome at a designated world is the event whose χ_e the world satisfies, by exclusivity. So every outcome realised has its child in π, and the invariant holds for the child. At a leaf the goal, in $\mathcal{L}^+$, holds at the designated worlds of t_β and so at those of t'_β.

At an action executed without transparency the products may differ: $t'_\beta \otimes \mathcal{E}$ may keep a pair (w, e) where μ holds in t'_β but not in t_β. The designated pairs still correspond, by exclusivity and applicability at t_β, and carry the same valuations. Everything evaluated after the action on β is propositional by assumption, and so depends only on the designated worlds' valuations, which are those of t_β's designated worlds descending from $[\![\varphi]\!]$. □

Proposition 1 (News can break a policy)**.** Without transparency, Theorem 1 fails, even for news true at every designated world.

Proof. Two agents a, b and atoms p, q; four worlds $u_{pq}, u_{p\bar{q}}, u_{\bar{p}q}, u_{\bar{p}\bar{q}}$. Agent a tells the worlds apart by p where q holds and not where it fails: its classes are $\{u_{pq}\}$, $\{u_{\bar{p}q}\}$ and $\{u_{p\bar{q}}, u_{\bar{p}\bar{q}}\}$. Agent b considers all four possible. The designated worlds are u_{pq} and $u_{\bar{p}q}$. The policy is a message from a

about p, events $p \wedge Kw_a p$ and $\neg p \wedge Kw_a p$, followed by a step whose precondition is $K_b q$. From s the message removes the two worlds where a does not know whether p, both with $\neg q$; at either outcome b's class keeps only worlds with q, and $K_b q$ holds. Announce p, true at u_{pq}: the model keeps u_{pq} and $u_{p\bar{q}}$, where a now knows p in both. The message is applicable and removes nothing, b's class after it is $\{u_{pq}, u_{p\bar{q}}\}$, and $K_b q$ fails. The message was not transparent at s: $Kw_a p$ failed at two worlds. `tools/policy_check.py` replays this example. □

The proposition is a hazard for any executor that keeps a policy across news: the news did not contradict anything the policy assumed, and the policy fails because an inference it relied on, from the occurrence of an action to the facts that made it possible, no longer goes through. In the three problems here every message the policies send, before the news and after, is transparent, which the check of Table 1 verifies at every state reached.

5.3 Planning again never costs depth

Corollary 1. Under the conditions of Theorem 1, for the policy π_t remaining at the state t where the news arrives, $\mathrm{h}^*(t|\varphi) \leq \mathrm{h}_{t|\varphi}(\pi_t) \leq \mathrm{h}_t(\pi_t)$.

Proof. π_t is a solution from t, and by Theorem 1 from $t|\varphi$, where its branches are among those realised from t. □

The inequality can be strict, and is in every act (Table 1). It concerns height, the worst branch, and says nothing of the others: a policy of least height may take any route on a branch shorter than its worst.

Remark 1 (The tautology)**.** $s|\top = s$, so announcing a tautology changes no model, and the policy planned after it is a policy of least height from the state the mission reached. The difference between *null* and *keep* measures what planning again is worth with nothing learned: the re-optimisation of branches the first policy was free to leave long because they were not its worst. The difference between *replan* and *null* measures the news.

5.4 Depth is not time: committing late

Corollary 1 bounds the worst branch's length, and says nothing of when a robot moves. A first batch, run without the step below, showed what that leaves open. In act 3 the policy planned after the cleaner's message has height 6 against the kept policy's 7, and puts the courier's drive to the west door before the cleaner speaks: the drive is on the worst branch, so it costs that branch nothing to start early. In the worlds where the east door is open the cleaner says so 2.5 s later, but the executor finishes the drive before it reads the answer, and the courier then drives back east. Planning again on the news was 18.9 s slower than keeping the policy on average in act 3, and 128 s slower in those worlds (Table 2). The same slack, in the policies planned at the start, is what made the tautology control look good in act 2.

Proposition 2 (Committing late)**.** Let π be a solution from s with a node whose action a is a drive of the courier, public and ontic, followed by a look or a message b of another robot whose preconditions and events mention none of the atoms a changes. Let π' move a below b into each of b's branches, and drop it on a branch where the courier's next action is a drive to a place other than a's target and the place it stands. Then π' is a solution from s and $\mathrm{h}_s(\pi') \leq \mathrm{h}_s(\pi)$.

Proof. A public ontic action has a single event that every agent observes, so its product keeps every relation and changes only the valuation, on the atoms of its effects. Since b's preconditions and events mention none of those atoms, b is applicable and has the same outcomes before a as after it, and the two products commute up to isomorphism. On a branch where the courier's next action is a drive g to another place, nothing between reads a's atoms, g's precondition holds with or without a since its target differs from both a's and the courier's place, and g's effects set the

	as planned				committed late			
act	replan	tautology	kept	saved	replan	tautology	kept	saved
1	168.8	197.7	195.4	+26.6	169.0	197.5	196.4	+27.4
2	147.2	161.4	175.4	+28.2	147.2	163.8	162.8	+15.6
3	193.0	162.2	174.1	-18.9	160.7	165.2	162.9	+2.2

Table 2: Mean mission time in simulated seconds, with the policies executed as AO* returns them (the first batch) and committed late (the batch of Section 6); saved: the paired mean of the kept policy's time minus the replanned one's. The classical runs are common to both.

courier's place whatever it was; the states after g coincide. Every branch of π' has the length of the corresponding branch of π, or one less. □

The mission applies this to every policy of every epistemic formulation, moving a drive past the other robots' looks and messages until it meets the courier's next action, and no further than another robot's drive, which the executor runs beside it anyway (`tools/lazy.py`). `tools/policy_check.py` confirms, for the policies before and after each news, that the result is a solution of the same height. In act 1 the courier is alone and nothing moves; in act 2 the courier waits for the scout's look through the hatch before setting out; in act 3, after the news, it waits for the cleaner's answer.

5.5 News invisible to replanning over facts

Definition 7 (Replanner over facts)**.** Let $\mathrm{val}(s) = \{V(w) : w \in W_d\}$. A replanner over facts is any procedure whose decision at a state depends on the state only through $\mathrm{val}(s)$.

A classical planner given the facts known, an optimistic one choosing a convenient valuation, a determinisation that samples one, and FF-Replan are all replanners over facts.

Proposition 3. If $\mathrm{val}(s|\varphi) = \mathrm{val}(s)$, every replanner over facts decides the same at $s|\varphi$ as at s.

The proof is the definition. The news of act 3 is such a formula: the announcement of $Kw_k open_e$ removes the seven worlds where the cleaner did not see the door, each the twin of a remaining world with the same valuation, so val is unchanged, while h^* drops from 7 to 6 because the new policy can ask k. News about what an agent knows can be worth something to a team that plans over knowledge and nothing to any planner over facts.

The other two kinds are visible. The literal of act 1 removes valuations, and a classical replanner uses it fully. The relation of act 2 removes valuations too; a replanner over the set of valuations uses it, but only by acting on a guess, since it cannot plan to look: the optimistic one takes the shortest door consistent with the relation and learns the relation's consequences only by driving into a shut door.

6 The batch

32 seeds, 9, 9 and 14 in the three acts, cycle through the valuations in which each act's news is true, three times in acts 1 and 2 and twice in act 3, and perturb every robot's start by up to 5 cm and 5 degrees. Each seed runs under all four formulations, 128 runs in all, headless, up to six side by side in lanes of their own. A run is judged by where the courier ended, by its own odometry, within 0.5 m of the resident's bedside, and by the executor's result; its time is simulated seconds from the mission's start.

Every epistemic run delivered, and none met a shut door: the courier crosses only a door it knows to be open (Table 3). In act 1 planning again on the news took 169.0 s on average against 196.4 s for the policy kept, and was faster in all 9 pairs (paired mean 27.4 s, 95 % interval 25.9 to 28.9 s, Wilcoxon $p = 0.004$; Table 4): the door panel's report turns the courier west mid-drive,

act	formulation	delivered	mean	sd	worst	drove	shut met
1	replan on the news	9/9	169.0	53.8	245.4	32.8	0 (0)
1	replan on a tautology	9/9	197.5	54.8	276.4	37.2	0 (0)
1	policy kept	9/9	196.4	54.6	271.3	37.2	0 (0)
1	classical replanning	9/9	175.3	66.8	267.5	34.5	3 (3)
2	replan on the news	9/9	147.2	8.9	156.5	28.8	0 (0)
2	replan on a tautology	9/9	163.8	0.6	164.7	29.9	0 (0)
2	policy kept	9/9	162.8	3.1	168.9	29.9	0 (0)
2	classical replanning	9/9	175.1	68.3	267.9	34.5	3 (3)
3	replan on the news	14/14	160.7	9.9	172.9	29.1	0 (0)
3	replan on a tautology	14/14	165.2	4.5	172.1	29.8	0 (0)
3	policy kept	14/14	162.9	5.2	172.2	29.8	0 (0)
3	classical replanning	14/14	201.7	93.7	372.8	39.5	8 (6)

Table 3: The batch. Delivered: runs with the courier at the resident and the executor's success. Times in simulated seconds over the runs delivered; the courier's drive in metres. Shut met: crossings begun through a door that proved shut, and the runs with any.

act	paired difference	pairs	mean	95 % interval	longer/equal/shorter	p
1	kept − replanned	9	+27.4	[25.9, 28.9]	9/0/0	0.004
1	tautology − news	9	+28.5	[24.6, 32.3]	9/0/0	0.004
1	kept − tautology	9	-1.1	[-3.6, 1.5]	3/0/6	0.250
1	classical − replanned	9	+6.2	[-4.3, 16.8]	3/2/4	0.652
2	kept − replanned	9	+15.6	[8.8, 22.4]	9/0/0	0.004
2	tautology − news	9	+16.6	[9.9, 23.4]	9/0/0	0.004
2	kept − tautology	9	-1.1	[-3.2, 1.1]	1/2/6	0.195
2	classical − replanned	9	+27.9	[-22.0, 77.9]	3/0/6	0.910
3	kept − replanned	14	+2.2	[-3.0, 7.5]	3/3/8	0.363
3	tautology − news	14	+4.5	[-1.4, 10.4]	10/2/2	0.058
3	kept − tautology	14	-2.3	[-5.0, 0.4]	2/0/12	0.078
3	classical − replanned	14	+41.0	[-12.5, 94.4]	6/0/8	0.326

Table 4: Paired differences in mission time, by seed; positive means the first formulation took longer. Equal: within 0.5 s. p: Wilcoxon signed-rank.

where the policy kept drives on to the east door, looks, and turns back. In act 2 the relation between doors saved 15.6 s (8.8 to 22.4 s, $p = 0.004$), again in every pair: with the scout's look through the hatch it settles the east door, and nobody drives to look at it.

In act 3 the cleaner's message saved 2.2 s on average (-3.0 to 7.5 s, $p = 0.363$), faster in 3 pairs of 14 and slower in 8, by a few seconds each. All of the gain is in the world where the middle and east doors are both shut (Table 5): the answer sends the courier west at once, 140.9 s against 163.4 s. Where the middle door is open, the scout's look has already decided the courier's way and the east door no longer matters; where only the east door is open, the answer spares a look of four seconds and costs the wait for it, and the halt. The news lowers the worst case from seven actions to six (Corollary 1), and in this residence the worst case is not where the time goes.

Planning again on a tautology took 1.1, 1.1 and 2.3 s longer than keeping the policy: with every policy committed late, planning again gains nothing when nothing is learned, and costs the halt. Classical replanning cannot use the third kind of news at all (Proposition 3). Where its guess about the west door is right it is the fastest formulation, 129.5 s on average against 146.6 s for planning again on the news in the same worlds, since it never stops to look; where the west door is shut it drives into the door, backs out and guesses again, 262 to 373 s, and it met 14 shut doors in 12 of its 32 runs.

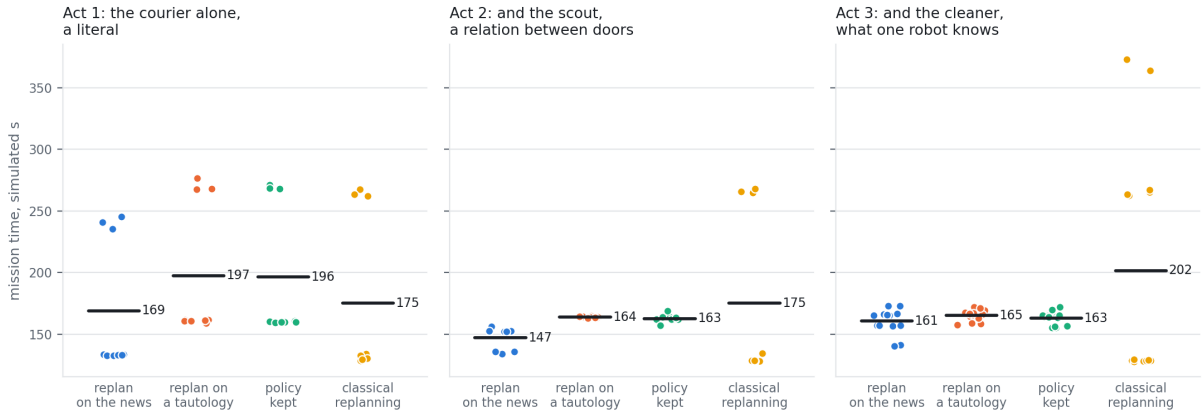


Figure 2: Mission time per run, by act and formulation; each point a run, the bar the mean. The four formulations of a seed start from the same world and poses.

act	doors	replan	tautology	kept	classical	shut met
1	west open, middle open, east shut	133.2	160.7	160.0	129.4	0
1	west open, middle shut, east shut	133.1	161.2	160.0	132.1	0
1	west shut, middle open, east shut	240.7	270.5	269.2	264.2	3
2	west open, middle open, east open	153.7	163.9	161.5	128.5	0
2	west open, middle shut, east shut	135.4	163.6	162.0	130.7	0
2	west shut, middle open, east open	152.4	164.0	164.8	266.1	3
3	west open, middle open, east open	165.6	169.6	164.8	128.8	0
3	west open, middle open, east shut	169.0	165.7	167.0	128.6	0
3	west open, middle shut, east open	157.1	161.6	156.1	128.4	0
3	west open, middle shut, east shut	140.9	166.9	163.4	128.5	0
3	west shut, middle open, east open	169.0	164.3	168.6	263.9	2
3	west shut, middle open, east shut	166.4	170.2	164.8	265.1	2
3	west shut, middle shut, east open	157.1	158.2	156.1	368.3	4

Table 5: Mean mission time by world, in simulated seconds, and the shut doors the classical formulation met.

The interruption. The news stopped the running drive at the executor's next tick, within the 0.1 s resolution of the record, and the new policy was ready 0.2 s of simulated time after the news. Planning again took 0.10 to 0.28 s of wall time, against up to 4.7 s for the first policy of a run, planned over more worlds.

7 Implementation findings

The subprocess planner plans from the start ePlanSys's `AletheiaPlanSolver`, which runs Aletheia as a command and is what the earlier demonstrations use, reads the task file and not the model the state node publishes; a replan with it plans from the initial state and forgets what the mission learned, the news included. The in-process `EpistemicPlanSolver` reads the published model, and is used here.

A baseline taken at the first tick Each `CheckBeliefUnchanged` takes the belief version it compares against when it is first ticked. News that arrives while no action runs, between one action's end and the next one's first tick, becomes the next action's baseline and interrupts nothing; the policy then runs on, still a solution under Theorem 1, but unaware. Every run here delivers its news mid-drive. Taking the baseline from the version the policy was planned against would close the window.

No terminal status for a halted action When the reactive sequence halts an action, the ex-

ecutor's `/action_execution_info` reports no cancellation, and the action stays *executing* there; the result of the plan carries it as cancelled.

A parameter array outliving its parameter The drive's first version looped over a parameter's string array fetched in the loop's own header; the array belongs to a temporary parameter destroyed before the loop runs, and every drive aborted on its first tick.

8 Limitations

The news is always true and always consistent with the model: a public announcement can only remove worlds, and ePlanSys refuses a formula false at every designated world. News that contradicts the model, a door the model took to be always open reported shut, needs belief revision or a model widened by regrounding, and is outside this report.

Aletheia minimises height, the number of actions on the worst branch, while the batch measures time. Corollary 1 bounds the first; the second depends on the residence's distances and on which of the policies of least height the search returns. Committing late (Proposition 2) removes one way the two part, a drive started before the information that decides it; it does not make the search minimise time, and it moves only drives past looks and messages.

Each run carries one piece of news at one instant, 14.0 s in, chosen to fall mid-drive. The worlds are cycled, not sampled, and the comparison is of formulations on the same worlds and starts, not of a population of residences. The robots are simulated, the doors' states are read from scans over a known layout, and the odometry is the simulator's.

References

[1] AWS RoboMaker. Small House World. https://github.com/aws-robotics/aws-robomaker-small-house-world, MIT-0 licence.

[2] A. Baltag, L. S. Moss, S. Solecki. The logic of public announcements, common knowledge, and private suspicions. *TARK*, 1998.

[3] H. van Ditmarsch, W. van der Hoek, B. Kooi. *Dynamic Epistemic Logic*. Springer, 2007.

[4] J. Plaza. Logics of public communications. *Synthese* 158(2):165–179, 2007 (first published 1989).

[5] T. Bolander, M. B. Andersen. Epistemic planning for single- and multi-agent systems. *Journal of Applied Non-Classical Logics* 21(1):9–34, 2011.

[6] M. B. Andersen, T. Bolander, M. H. Jensen. Conditional epistemic planning. *JELIA*, LNCS 7519, 2012.

[7] H. van Ditmarsch, B. Kooi. The secret of my success. *Synthese* 151(2):201–232, 2006.

[8] S. Yoon, A. Fern, R. Givan. FF-Replan: a baseline for probabilistic planning. *ICAPS*, 2007.

[9] A. Coles, A. Coles, M. Fox, D. Long. Forward-chaining partial-order planning. *ICAPS*, 2010.

[10] F. Martín, J. Ginés Clavero, V. Matellán, F. J. Rodríguez. PlanSys2: a planning system framework for ROS2. *IROS*, 2021.